also uses a Proc as a *Clean Room (87)* to execute the code in a separate scope. (Note that the new value of $SAFE applies only inside the Proc. Contrary to what happens with other global variables, the Ruby interpreter takes care to reset $SAFE to its former value after the call.)

"Now," Bill says, finally wrapping up his long explanation, "you know about eval and how dangerous it can be. But eval is great to get code up and running quickly. That's why you can use this method as a first step to solve your original problem: writing the attribute generator for the boss."

Kernel#eval() and Kernel#load()

Ruby has methods like Kernel#load and Kernel#require that take the name of a source file and execute code from that file. If you think about it, evaluating a file is not that different from evaluating a string. This means load and require are somewhat similar to eval. Although these methods are not really part of the *eval family, you can think of them as first cousins.

You can usually control the content of your files, so you don't have as many security concerns with load and require as you do with eval. Still, safe levels higher than 1 do put some limitations on importing files. For example, a safe level of 2 or higher prevents you from using load with a tainted filename.

Quiz: Checked Attributes (Step 1)

Where you take your first step toward solving the boss' challenge, with Bill looking over your shoulder.

You and Bill look back at the first two steps of your development plan:

1. Write a *Kernel Method (32)* named add_checked_attribute using eval to add a super-simple validated attribute to a class.
2. Refactor add_checked_attribute to remove eval.

Focus on the first step. The add_checked_attribute method should generate a reader method and a writer method, pretty much like attr_accessor does. However, add_checked_attribute should differ from attr_accessor in three ways. First, while attr_accessor is a *Class Macro (117)*, add_checked_attribute is supposed to be a simple *Kernel Method (32)*. Second, attr_accessor is written in C, while add_checked_attribute should use plain Ruby (and a *String of Code (141)*). Finally, add_checked_attribute should add one basic example of validation to the attribute: the attribute will raise a runtime exception if you assign it either nil or false. (You'll deal with flexible validation down the road.)

These requirements are expressed more clearly in a test suite: